

De Hibernate/Spring Boot a Entity Framework Core / ASP.NET Core

Guía complementaria — Preparación Espiral MS

Julio 2026

Guía de aprendizaje complementaria al proyecto MarinaApi

Qué es este documento: una traducción didáctica, capítulo a capítulo, de tu tutorial de acceso a datos en Java (Hibernate → Spring Boot → REST) a su equivalente en C#/.NET. No es solo código: es la explicación del **por qué**, igual que tus capítulos originales.

Complementa al código: todo lo que se explica aquí ya está implementado y funcionando en el proyecto MarinaApi.zip que te generé antes. Cuando un apartado diga “ver archivo del proyecto”, ese archivo ya existe en el zip — este documento es la clase teórica; el zip es la práctica ya resuelta.

Requisito previo: este documento asume que ya has hecho las Lecciones 1-8 de tu GUIA_DEFINITIVA_CSHARP.md (sintaxis C#, LINQ, async/await, generics, patrones de diseño, EF Core básico). Aquí profundizamos específicamente en la capa de acceso a datos y API REST, con el mismo dominio que tu tutorial Java (Barco, Amarre, Regata).

Índice

1. [Introducción: Hibernate vs Entity Framework Core](#)
 2. [Configuración del proyecto](#)
 3. [Entidades y Mapeo](#)
 4. [Relaciones entre Entidades](#)
 5. [Estados y Change Tracking](#)
 6. [Repositorio Genérico \(vs DAO\)](#)
 7. [Consultas con LINQ \(vs HQL/Criteria\)](#)
 8. [Testing con xUnit y Moq \(vs JUnit/Mockito\)](#)
 9. [ASP.NET Core Web API \(vs Spring Boot\)](#)
 10. [Capa de Servicio](#)
 11. [API REST con Controllers](#)
 12. [DTOs y Mapper](#)
 13. [Swagger / OpenAPI](#)
 14. [Probar la API: Swagger UI y Postman](#)
 15. [Resumen: mapa completo del proyecto](#)
-

1. Introducción: Hibernate vs Entity Framework Core

[↑ Volver al índice](#)

1.1. Mismo problema, dos soluciones

Ya conoces el problema desde tu Capítulo 1 de Java: el **desajuste objeto-relacional**. Tus clases C# (Barco, Amarre, Regata) no encajan de forma natural con las tablas SQL. **Entity Framework Core (EF Core)** resuelve exactamente lo mismo que Hibernate, siendo el ORM oficial de Microsoft para .NET.

Concepto	Java	C#
Especificación	JPA	No existe una “especificación” separada — EF Core es a la vez la interfaz y la implementación
Implementación ORM	Hibernate	Entity Framework Core
Motor de BD (Espiral MS)	MySQL (en tu tutorial)	SQL Server
Gestor de sesión/contexto	Session / SessionFactory	DbContext
Colección de entidades	session.createQuery("from Barco")	DbSet<Barco>

Vocabulario: en Java, JPA es la “receta” y Hibernate el “cocinero” (Cap. 1.4-1.5). En C# **no existe esa separación**: EF Core es receta y cocinero a la vez. Esto simplifica las cosas — no hay que elegir “implementación”, solo instalar el paquete NuGet del proveedor de tu base de datos (Microsoft.EntityFrameworkCore.SqlServer).

1.2. El concepto central: DbContext

Donde Java tenía SessionFactory + Session, C# tiene una única clase: el DbContext. Es, a la vez: - El registro de qué entidades existen (como hibernate.cfg.xml) - La fábrica de sesiones (como SessionFactory) - La sesión de trabajo en sí (como Session)

```
// Ver archivo del proyecto: Data/MarinaDbContext.cs
public class MarinaDbContext : DbContext
{
    public DbSet<Barco> Barcos => Set<Barco>();
    public DbSet<Amarre> Amarres => Set<Amarre>();
    public DbSet<Regata> Regatas => Set<Regata>();
}
```

Comparado con Java:

```
// Lo que hacía HibernateUtil + hibernate.cfg.xml + las clases @Entity, todo junto
SessionFactory sf = HibernateUtil.getSessionFactory();
Session session = sf.openSession();
```

TIP: en Java, tenías que abrir y cerrar la Session manualmente con try-with-resources en cada método DAO (Cap. 7.3.1). En C#, el DbContext se inyecta automáticamente por petición HTTP gracias a la Inyección de Dependencias de ASP.NET Core — nunca lo abres/cierras a mano.

1.3. Nuestro proyecto: Sistema de Gestión Marítima

Mantenemos exactamente el mismo dominio que tu tutorial Java, para que puedas comparar directamente:

- **Barco:** nombre, eslora, tipo, manga, capacidad. Tiene un amarre (1:1) y participa en regatas (N:M).
- **Amarre:** ubicación, precio, profundidad, electricidad. Pertenece a un barco (1:1).

- **Regata:** nombre, lugar, fecha, distancia. Tiene varios barcos inscritos (N:M).
- **Checkpoint:** antes de continuar, entiende que **EF Core = JPA + Hibernate fusionados en una sola herramienta**, y que DbContext sustituye a Session + SessionFactory + hibernate.cfg.xml.

2. Configuración del proyecto

[↑ Volver al índice](#)

2.1. El equivalente al pom.xml: .csproj

```
<!-- Ver archivo del proyecto: MarinaApi.csproj -->
<ItemGroup>
  <PackageReference Include="Microsoft.EntityFrameworkCore" Version="8.0.8" />
  <PackageReference Include="Microsoft.EntityFrameworkCore.SqlServer" Version="8.0.8" />
  <PackageReference Include="Microsoft.EntityFrameworkCore.Design" Version="8.0.8" />
  <PackageReference Include="Swashbuckle.AspNetCore" Version="6.6.2" />
</ItemGroup>
```

Java (pom.xml)	C# (.csproj)
<dependency>	<PackageReference>
Repositorio Maven Central	Repositorio NuGet
mvn install	dotnet restore

2.2. El equivalente a application.properties: appsettings.json

```
// Ver archivo del proyecto: appsettings.json
{
  "ConnectionStrings": {
    "MarinaDb": "Server=localhost,1433;Database=gestion_maritima;User Id=sa;Password=TuPassw"
  }
}
```

Comparado con Java:

```
spring.datasource.url=jdbc:mysql://localhost:3306/gestion_maritima?...
spring.datasource.username=root
spring.datasource.password=root
```

CUIDADO: el proyecto usa **SQL Server**, no MySQL como el tutorial Java original — es el motor real de Espiral MS. Si alguna vez ves ejemplos en internet con UseMySQL(...), no los copies directamente: nuestro Program.cs usa UseSqlServer(...).

2.3. Program.cs: el equivalente a la autoconfiguración de Spring Boot

Este es probablemente el cambio más grande de mentalidad viniendo de Spring Boot. En Java, @SpringBootApplication escaneaba automáticamente tus clases @Service, @Repository, @RestController (Cap. 10.3: “Spring Boot examina las dependencias del pom.xml y configura los beans automáticamente”).

En ASP.NET Core no hay escaneo automático. Cada servicio se registra explícitamente:

```
// Ver archivo del proyecto: Program.cs
builder.Services.AddScoped<IBarcoRepository, BarcoRepository>();
builder.Services.AddScoped<IBarcoService, BarcoService>();
```

Java	C#
@Service detectado automáticamente	builder.Services.AddScoped<Interfaz, Implementacion>() explícito
@Autowired en el campo	Inyección por constructor (obligatoria, no hay alternativa por campo)
spring.jpa.hibernate.ddl-auto=update	dotnet ef migrations add + dotnet ef database update

Buena práctica: el registro explícito de C# es más verboso, pero también más trazable — con un vistazo a Program.cs ves *todas* las dependencias de la aplicación en un solo sitio, sin tener que buscar anotaciones desperdigadas por el código.

2.4. Docker Compose para SQL Server

```
# Ver archivo del proyecto: docker-compose.yml
services:
  sqlserver:
    image: mcr.microsoft.com/mssql/server:2022-latest
    environment:
      ACCEPT_EULA: "Y"
      MSSQL_SA_PASSWORD: "TuPassword123!"
    ports:
      - "1433:1433"
```

Exactamente el mismo concepto que el docker-compose.yml de MySQL de tu Capítulo 10, cambiando solo la imagen y el puerto (1433 en vez de 3306).

2.5. Ejercicio

1. Clona el proyecto del zip, levanta docker compose up -d
 2. Ejecuta dotnet ef migrations add InitialCreate y observa el archivo de migración generado — compáralo con el SQL que Hibernate mostraba en consola en tu Cap. 4.9
 3. Ejecuta dotnet ef database update y verifica con SQL Server Management Studio (o Azure Data Studio) que las tablas Barcos, Amarres, Regatas y BarcoRegata se han creado
- **Checkpoint:** debes entender que en C# **no hay autoconfiguración mágica** como en Spring Boot — todo se declara explícitamente en Program.cs, y eso es una decisión de diseño, no una carencia.

3. Entidades y Mapeo

[↑ Volver al índice](#)

3.1. @Entity no existe en EF Core — y eso es intencional

Este es el cambio conceptual más importante del capítulo. En Java, cada clase persistente **necesita** la anotación @Entity (Cap. 4.2.1). En EF Core, **cualquier clase C# que esté declarada**

como `DbSet<T>` en el `DbContext` ya es una entidad, sin anotarla:

```
// Ver archivo del proyecto: Models/Barco.cs
public class Barco
{
    public long Id { get; set; }
    public string Nombre { get; set; } = string.Empty;
    public string Tipo { get; set; } = string.Empty;
    public int Eslora { get; set; }
    public int Manga { get; set; }
    public int Capacidad { get; set; }
}
```

Comparado con Java:

```
@Entity          // ← EF Core no necesita esto
@Data            // ← EF Core no necesita Lombok, las propiedades ya son "auto"
@NoArgsConstructor // ← EF Core no necesita constructor vacío explícito
public class Barco {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    // ...
}
```

3.2. Convención sobre configuración

EF Core usa **convenciones de nombres** para inferir el mapeo sin anotaciones:

Convención EF Core	Equivalente explícito Java
Propiedad llamada Id (o BarcoId) Tipo numérico (int, long) en Id	@Id @GeneratedValue(strategy = GenerationType.IDENTITY) — EF Core lo asume automáticamente en SQL Server
Nombre de la clase Barco	Nombre de tabla Barcos (EF Core pluraliza sola)
Nombre de propiedad Nombre	Nombre de columna Nombre

TIP: esto es el principio de “**convención sobre configuración**”: si sigues los nombres estándar, no necesitas anotar nada. Solo anotas (o configuras vía Fluent API) cuando te **desvías** de la convención — igual que en Java solo usabas `@Column(name=...)` cuando el nombre por defecto no te servía (Cap. 4.2.4).

3.3. Cuándo SÍ necesitas anotar: Data Annotations

Para las validaciones que en Java hacías con `@Column(nullable = false, length = 100)`:

```
using System.ComponentModel.DataAnnotations;

public class Barco
{
    [Required, MaxLength(100)]
    public string Nombre { get; set; } = string.Empty;
}
```

Java (@Column)	C# (DataAnnotations)
nullable = false	[Required]
length = 100	[MaxLength(100)]
unique = true	Se configura en el DbContext (HasIndex().IsUnique()), no con atributo

3.4. Lombok no existe — y no hace falta

Java necesitaba Lombok (@Data, @NoArgsConstructor) para no escribir 60 líneas de getters/setters (Cap. 4.5). En C#, las **propiedades automáticas** (public string Nombre { get; set; }) ya generan el getter y setter en una sola línea — es sintaxis nativa del lenguaje, no una librería externa.

```
// Esto YA incluye getter y setter, sin Lombok, sin anotación
public string Nombre { get; set; } = string.Empty;
```

¿Sabías que? El = string.Empty al final no es capricho: con *nullable reference types* activado (que ya usas desde la Lección 1 de tu guía de C#), el compilador exige que toda propiedad string no anulable tenga un valor inicial. Es el equivalente a que Hibernate necesitara un constructor vacío — pero resuelto por el compilador, no por una librería.

3.5. @Temporal → DateOnly

Java necesitaba @Temporal(TemporalType.DATE) porque java.util.Date incluye hora y zona horaria por defecto. C# moderno (desde .NET 6) tiene un tipo dedicado que resuelve esto de raíz:

```
// Ver archivo del proyecto: Models/Regata.cs
public DateOnly Fecha { get; set; } // Solo fecha, sin hora – no hace falta anotación

@Temporal(TemporalType.DATE) // Necesario en Java: Date por defecto lleva hora
private Date fecha;
```

3.6. @Enumerated → enum de C

Si tuvieras un campo de tipo fijo (Java Cap. 4.4.2):

```
public enum TipoBarco { Velero, Motor, Catamaran }

public TipoBarco Tipo { get; set; }
// EF Core lo guarda como int por defecto (como EnumType.ORDINAL)
// Para guardarlo como texto (como EnumType.STRING, más seguro):
// modelBuilder.Entity<Barco>().Property(b => b.Tipo).HasConversion<string>();
```

CUIDADO: igual que en Java con EnumType.ORDINAL (Cap. 4.4.2), si guardas el enum como número y luego reordenas sus valores, los datos existentes se corrompen. La recomendación es la misma: convierte a string explícitamente en el DbContext.

3.7. Ejercicio

1. Abre Models/Barco.cs, Models/Amarre.cs y Models/Regata.cs del proyecto
2. Por cada propiedad, identifica cuál sería su anotación JPA equivalente en Java (aunque en C# no la necesites escribir)
3. Añade una nueva entidad Tripulante (nombre, rol, BarcoId) siguiendo la misma convención, sin copiar ninguna anotación de Java

- **Checkpoint:** debes entender por qué EF Core necesita **menos código visible** que Hibernate para el mismo resultado: las convenciones de nombres + las propiedades automáticas de C# cubren lo que en Java hacían @Entity, @Id, @GeneratedValue y Lombok juntos.

4. Relaciones entre Entidades

[↑ Volver al índice](#)

4.1. mappedBy no existe en EF Core

Este es el cambio más importante de todo el capítulo de relaciones. En Java, marcabas el “lado inverso” de una relación con mappedBy **dentro de la propia entidad** (Cap. 5.1.2). En EF Core, **las relaciones se configuran fuera de las entidades**, en el DbContext, con Fluent API:

```
// Ver archivo del proyecto: Data/MarinaDbContext.cs
protected override void OnModelCreating(ModelBuilder modelBuilder)
{
    modelBuilder.Entity<Amarre>()
        .HasOne(a => a.Barco)           // Amarre tiene UN Barco
        .WithOne(b => b.Amarre)         // Barco tiene UN Amarre
        .HasForeignKey<Amarre>(a => a.BarcoId) // Amarre es el propietario (tiene la FK)
        .OnDelete(DeleteBehavior.Cascade);
}
```

Comparado con Java:

```
// Lado propietario (Amarre) – DENTRO de la clase
@OneToOne
@JoinColumn(name = "barco_id")
private Barco barco;

// Lado inverso (Barco) – DENTRO de la otra clase
@OneToOne(mappedBy = "barco", cascade = CascadeType.ALL)
private Amarre amarre;
```

Concepto	Java	C#
Dónde se declara la relación	Dentro de cada entidad, con anotaciones	Fuera, centralizado en OnModelCreating
Entidad propietaria (tiene la FK)	Sin mappedBy	.HasForeignKey<T>() indica cuál
Lado inverso	Con mappedBy = "..."	.WithOne() / .WithMany()
Cascada	cascade = CascadeType.ALL	.OnDelete(DeleteBehavior.Cascade)

Buena práctica: tener toda la configuración de relaciones **en un solo archivo** (MarinaDbContext.cs) en vez de repartida entre varias clases con anotaciones es una ventaja real de EF Core: cuando algo no cuadra en las relaciones de tu BD, sabes exactamente dónde mirar.

4.2. Relación 1:1 completa

```
// Models/Barco.cs – solo la propiedad de navegación, SIN anotaciones de relación
public Amarre? Amarre { get; set; }
```

```
// Models/Amarre.cs – con la FK explícita
public long? BarcoId { get; set; }
public Barco? Barco { get; set; }
```

El ? en Amarre? Amarre es importante: en Java, un objeto puede ser null sin que el compilador te avise (Cap. 1, *NullPointerException*). En C# con *nullable reference types*, declaras explícitamente que esa relación **puede no existir** — el compilador te obliga a comprobarlo antes de usarla.

4.3. Relación N:M — la mayor simplificación de EF Core

Aquí está la diferencia más grande respecto a Java. En tu Cap. 5.4, para la relación N:M Barco↔Regata necesitabas:

```
@ManyToMany
@JoinTable(
    name = "barco_regata",
    joinColumns = @JoinColumn(name = "barco_id"),
    inverseJoinColumns = @JoinColumn(name = "regata_id")
)
private List<Regata> regatas;
```

En C# con EF Core 5+ (**skip navigations**), la tabla intermedia se genera sola con una sola línea en el DbContext, y las entidades ni siquiera necesitan saber que existe:

```
// Models/Barco.cs
public List<Regata> Regatas { get; set; } = new();

// Models/Regata.cs
public List<Barco> Barcos { get; set; } = new();

// Data/MarinaDbContext.cs – la ÚNICA configuración que hace falta
modelBuilder.Entity<Barco>()
    .HasMany(b => b.Regatas)
    .WithMany(r => r.Barcos)
    .UsingEntity(j => j.ToTable("BarcoRegata"));
```

¿Sabías que? No existe ninguna clase C# BarcoRegata.cs en el proyecto — EF Core gestiona la tabla intermedia internamente, sin que tengas que modelarla como entidad, a diferencia de otros ORMs donde sí haría falta una clase explícita para la tabla de unión.

4.4. Cascade — misma idea, sintaxis distinta

CascadeType de Java	DeleteBehavior de C#
CascadeType.REMOVE	DeleteBehavior.Cascade
CascadeType.ALL (sin remove)	DeleteBehavior.ClientCascade o configurarlo por tipo de operación
Sin cascade	DeleteBehavior.Restrict (por defecto en EF Core)

4.5. Ejercicio

1. Abre Data/MarinaDbContext.cs y localiza las dos relaciones configuradas

2. Genera la migración (`dotnet ef migrations add`) y busca en el archivo generado la tabla `BarcoRegata` — comprueba que tiene exactamente las mismas dos columnas FK que la tabla intermedia manual de Java (Cap. 5.4.1)
3. Añade una relación 1:N nueva: `Regata` tiene un `Organizador` (nueva entidad con nombre y email), donde un organizador puede dirigir varias regatas
 - **Checkpoint:** entiendes que EF Core mueve la configuración de relaciones **fuera** de las entidades (a diferencia de Java, que la pone dentro con anotaciones), y que las relaciones N:M no necesitan una clase Java equivalente a la tabla intermedia.

5. Estados y Change Tracking

[↑ Volver al índice](#)

5.1. Los 4 estados de Hibernate vs el Change Tracker de EF Core

Java (Hibernate) distingue explícitamente 4 estados de una entidad: *Transient*, *Persistent*, *Detached*, *Removed*. EF Core simplifica esto con un **Change Tracker** interno que observa automáticamente los cambios:

Estado Hibernate	Equivalente en EF Core
Transient (recién creado con <code>new</code> , sin guardar)	<code>EntityState.Detached</code>
Persistent (gestionado por la sesión activa)	<code>EntityState.Unchanged</code> / <code>Modified</code> / <code>Added</code>
Detached (sesión cerrada)	<code>EntityState.Detached</code>
Removed	<code>EntityState.Deleted</code>

5.2. Cómo funciona en la práctica

```
var barco = await _context.Barcos.FindAsync(1); // Estado: Unchanged (recién leído)
barco.Nombre = "Nuevo nombre";               // Estado: Modified (EF Core lo detecta)
await _context.SaveChangesAsync();             // Genera el UPDATE, vuelve a Unchanged
```

No necesitas llamar a `session.update(barco)` como en Java. EF Core compara automáticamente los valores actuales contra los que leyó de la base de datos (usando una técnica llamada *snapshot tracking*) y genera el UPDATE solo con las columnas que realmente cambiaron.

```
// Java: tenías que llamar update() explícitamente
Transaction tx = session.beginTransaction();
session.update(barco); // ← paso manual obligatorio
tx.commit();
```

```
// C#: basta con modificar la propiedad y guardar
barco.Nombre = "Nuevo nombre";
await _context.SaveChangesAsync(); // EF Core ya sabía que había cambiado
```

TIP: esto es la razón por la que en `Services/BarcoService.cs` el método `UpdateAsync` no llama a ningún “update” del repositorio en el sentido Java — simplemente modifica las propiedades del objeto ya rastreado y llama a `SaveChangesAsync`.

5.3. `AsNoTracking()` — para consultas de solo lectura

Cuando sabes que **no** vas a modificar los datos (por ejemplo, un GET que solo lista), puedes decirle a EF Core que no gaste memoria rastreando cambios:

```
var barcos = await _context.Barcos.AsNoTracking().ToListAsync();
```

No existe un equivalente directo en Hibernate — es una optimización propia de EF Core.

5.4. Ejercicio

1. En un breakpoint dentro de `BarcoService.UpdateAsync`, inspecciona `_context.Entry(barco).State` antes y después de modificar una propiedad
 2. Compara mentalmente con el flujo Java: ¿en qué línea exacta de tu `BarcoDAOImpl.update()` (Cap. 7) ocurría el equivalente?
- **Checkpoint:** entiendes que EF Core detecta cambios automáticamente (Change Tracking) en vez de exigir una llamada explícita a `update()` como Hibernate.
-

6. Repositorio Genérico (vs DAO)

[↑ Volver al índice](#)

6.1. El problema que Java resolvía a medias

Tu Cap. 7 terminaba reconociendo el problema: `BarcoDAOImpl`, `AmarreDAOImpl` y `RegataDAOImpl` eran casi idénticos, y proponía un `GenericDAO<T>` — pero **sin implementación completa**, solo la interfaz (Cap. 7.4).

En el proyecto C#, ese `GenericDAO<T>` sí está completamente implementado y en uso real:

```
// Ver archivo del proyecto: Repositories/GenericRepository.cs
public class GenericRepository<T> : IGenericRepository<T> where T : class
{
    private readonly MarinaDbContext _context;
    private readonly DbSet<T> _dbSet;

    public async Task<T?> FindByIdAsync(long id, CancellationToken ct = default) =>
        await _dbSet.FindAsync(new object[] { id }, ct);

    public async Task<T> AddAsync(T entity, CancellationToken ct = default)
    {
        await _dbSet.AddAsync(entity, ct);
        await _context.SaveChangesAsync(ct);
        return entity;
    }
    // ... UpdateAsync, DeleteAsync, FindAllAsync
}
```

Una única clase sirve para Barco, Amarre y Regata a la vez — en vez de las ~450 líneas duplicadas entre los 3 `DAOImpl` de Java.

6.2. Extender el genérico con consultas propias

Igual que en Java `BarcoDAO` podía tener métodos propios además de los heredados de `GenericDAO<T>`, en C#:

```
// Ver archivo del proyecto: Repositories/BarcoRepository.cs
public interface IBarcoRepository : IGenericRepository<Barco>
{
    Task<Barco?> FindByNombreAsync(string nombre, CancellationToken ct = default);
    Task<List<Barco>> FindByTipoAsync(string tipo, CancellationToken ct = default);
}
```

Java	C#
interface BarcoDAO extends GenericDAO<Barco> Prefijo I no es obligatorio	interface IBarcoRepository : IGenericRepository<Barco> Prefijo I en interfaces sí es convención obligatoria (recuerda tu Lección 1)

6.3. try-with-resources no hace falta

En Java, cada método DAO abría su propia sesión con try (Session session = ...) (Cap. 7.3.1). En C#, el DbContext se inyecta una vez por petición HTTP (AddScoped en Program.cs) y **vive durante toda la petición** — no lo abres/cierras en cada método.

```
// Java: abrir sesión en CADA método
try (Session session = HibernateUtil.getSessionFactory().openSession()) {
    return session.find(Barco.class, id);
}
```

```
// C#: el DbContext ya está inyectado, listo para usar
public async Task<T?> FindByIdAsync(long id, CancellationToken ct = default) =>
    await _dbSet.FindAsync(new object[] { id }, ct);
```

6.4. Transacciones: implícitas vs explícitas

Java exigía el patrón manual beginTransaction() → operación → commit() en cada método de escritura (Cap. 7.3.2). En EF Core, SaveChangesAsync() **ya es una transacción implícita** por sí sola — si falla a mitad, no aplica ningún cambio.

Solo necesitas una transacción **explícita** cuando agrupas varias operaciones que deben tener éxito o fallar todas juntas (ver RegataService.DeleteAsync en el proyecto, que agrupa desvincular N:M + borrar en una sola transacción con BeginTransactionAsync()).

6.5. Ejercicio

1. Compara línea por línea Repositories/GenericRepository.cs con BarcoDAOImpl.java (Cap. 7.2.1): identifica qué método C# sustituye a cada bloque try (Session...) { Transaction tx = ...}
2. Añade un nuevo método FindByCapacidadGreaterThanAsync a IBarcoRepository y su implementación
 - **Checkpoint:** entiendes por qué el repositorio genérico de C# elimina la triplicación de código que Java solo diagnosticaba sin resolver del todo.

7. Consultas con LINQ (vs HQL/Criteria)

[↑ Volver al índice](#)

7.1. Una sola herramienta para las tres que usaba Java

Tu Cap. 8 explicaba **tres** formas distintas de consultar en Java: HQL, SQL nativo y Critería API, cada una con su sintaxis propia. En C#, **LINQ cubre los tres casos** con una única sintaxis (la misma que ya conoces de tu Lección 2):

```
// Equivalente a HQL: "FROM Barco WHERE tipo = :tipo"
await _context.Barcos.Where(b => b.Tipo == tipo).ToListAsync();

// Equivalente a Critería API (type-safe, sin strings)
await _context.Barcos.Where(b => b.Eslora > 10).ToListAsync();
```

Java	Equivalente C#
HQL ("FROM Barco WHERE ...", strings)	LINQ con lambdas — type-safe por defecto , sin strings
Critería API (CriteriaBuilder, Root<T>, muy verbosa)	LINQ — mismo type-safety, mucho menos código
SQL nativo (createNativeQuery)	FromSqlRaw() — para los casos raros que lo necesiten

Buena práctica: en Java tenías que **elegir** entre HQL (legible pero no type-safe) y Critería API (type-safe pero verbosa). En C#, LINQ te da **ambas ventajas a la vez** — no hay ese compromiso.

7.2. Parámetros — SQL Injection resuelto por diseño

Java necesitaba disciplina para usar setParameter() en vez de concatenar strings (Cap. 8.2.2, con aviso explícito de SQL Injection). En LINQ, **no existe la opción insegura**: como escribes expresiones C# (lambdas), no hay forma de “concatenar” un valor directamente en la consulta como en SQL con strings.

```
// Esto es LINQ compilado a expresión – EF Core lo traduce a parámetro SQL
// automáticamente, sin que puedas hacerlo "mal" ni queriendo
await _context.Barcos.Where(b => b.Nombre == nombreDelUsuario).ToListAsync();
```

7.3. JOIN FETCH → Include()

```
// Java (Cap. 8.2.4 y Cap. 11.4)
@Query("SELECT b FROM Barco b JOIN FETCH b.regatas WHERE b.id = :id")
Barco findByIdWithRegatas(@Param("id") Long id);

// Ver archivo del proyecto: Repositories/BarcoRepository.cs
public async Task<Barco?> FindByIdWithRegatasAsync(long id, CancellationToken ct = default)
    await _context.Barcos
        .Include(b => b.Regatas)
        .FirstOrDefaultAsync(b => b.Id == id, ct);
```

Include() (y ThenInclude() para relaciones anidadas) es el equivalente exacto de JOIN FETCH: carga la relación en la **misma** consulta SQL, evitando el problema N+1 — el mismo problema de rendimiento que tu Cap. 8 no llegaba a mencionar explícitamente pero que JOIN FETCH prevenía.

7.4. Consultas derivadas por nombre (Spring Data) → simplemente no hacen falta

Tu Cap. 11 mostraba cómo Spring Data generaba SQL a partir del **nombre del método** (findByEsloraGreaterThan). En C#, ese “truco” de nomenclatura no es necesario: **ya escribes directamente la expresión LINQ**, que es igual de concisa y además no depende de que aciertes con las palabras clave exactas (GreaterThan, Containing...) que Spring Data exige.

```
// Spring Data infiere el SQL del NOMBRE del método – mágico, pero rígido
List<Barco> findByEsloraGreaterThan(int eslora);

// C#: escribes la condición directamente, sin depender de convenciones de nombres
public Task<List<Barco>> FindByEsloraGreaterThanAsync(int eslora, CancellationToken ct) =>
    _context.Barcos.Where(b => b.Eslora > eslora).ToListAsync(ct);
```

7.5. Agregaciones y proyecciones

```
// COUNT
await _context.Barcos.LongCountAsync(b => b.Tipo == tipo);

// Proyección (como SELECT b.nombre, b.eslora FROM Barco b de HQL)
var resumen = await _context.Barcos
    .Select(b => new { b.Nombre, b.Eslora })
    .ToListAsync();
```

7.6. Ejercicio

1. Abre Repositories/BarcoRepository.cs y reescribe mentalmente cada método como si fuera HQL — comprueba que sabrías traducirlo en ambas direcciones
 2. Añade una consulta LINQ que obtenga el promedio de eslora de los barcos de un tipo dado (equivalente a `SELECT AVG(b.eslora) FROM Barco b WHERE b.tipo = :tipo` del Cap. 8.2.5)
- **Checkpoint:** entiendes que LINQ sustituye simultáneamente a HQL, Critería API y (en gran parte) a las consultas derivadas de Spring Data, con una sola sintaxis type-safe.

8. Testing con xUnit y Moq (vs JUnit/Mockito)

[↑ Volver al índice](#)

8.1. Correspondencia directa de conceptos

Tu Cap. 9 se traduce casi 1:1 — los conceptos son idénticos, solo cambia la sintaxis:

Java (JUnit 5 + Mockito)	C# (xUnit + Moq)
@Test	[Fact]
@BeforeEach	Constructor de la clase de test
@Mock	new Mock<T>()
@InjectMocks	Pasar mock.Object al constructor manualmente
when(x).thenReturn(y)	mock.Setup(x => ...).ReturnsAsync(y)

Java (JUnit 5 + Mockito)	C# (xUnit + Moq)
<code>verify(x).metodo()</code>	<code>mock.Verify(x => x.Metodo(), Times.Once)</code>
<code>assertEquals(a, b)</code>	<code>a.Should().Be(b) (FluentAssertions)</code>
<code>assertThrows(...)</code>	<code>await FluentAssertions.Awaiting(...).Should().ThrowAsync<T>()</code>

8.2. Ejemplo lado a lado

```
// Java (Cap. 9.6)
@Test
void testFindById() {
    Barco barcoEsperado = new Barco();
    barcoEsperado.setId(1L);
    when(session.find(Barco.class, 1L)).thenReturn(barcoEsperado);

    Barco resultado = barcoDAO.findById(1L);

    assertNotNull(resultado);
    assertEquals(1L, resultado.getId());
}
```

```
// Ver archivo del proyecto: MarinaApi.Tests/BarcoServiceTests.cs
[Fact]
public async Task FindByIdAsync_CuandoExiste_DevuelveDto()
{
    var barco = new Barco { Id = 1, Nombre = "Estrella del Mar" };
    _repositoryMock.Setup(r => r.FindByIdAsync(1, It.IsAny<CancellationToken>()))
        .ReturnsAsync(barco);

    var resultado = await _service.FindByIdAsync(1);

    resultado.Nombre.Should().Be("Estrella del Mar");
}
```

8.3. Una diferencia real: qué testeamos

En Java testeabas el **DAO** directamente (mockeando Session/Transaction). En el proyecto C#, testeamos el **Servicio** (mockeando el Repositorio), porque el repositorio genérico ya es tan delgado que apenas tiene lógica propia que merezca test — toda la lógica de negocio real vive en la capa de Servicio.

TIP: esto es coherente con la arquitectura en capas que ya conoces (Controller → Service → Repository, Cap. 12.1 de Java): testeas la capa que **decide**, no la que solo traduce llamadas a la base de datos.

8.4. It.IsAny<CancellationToken>()

Una diferencia sin equivalente directo en Java: como todos nuestros métodos son async con `CancellationToken`, los mocks necesitan indicar que aceptan **cualquier** token, ya que el valor exacto no importa para el test.

8.5. Ejercicio

1. Ejecuta `dotnet test` desde `MarinaApi.Tests/` y observa la salida — compárala mentalmente con la “barra verde” de IntelliJ (Cap. 9.7)
 2. Escribe un test nuevo `UpdateAsync_CuandoNoExiste_LanzaNotFoundException`, siguiendo el patrón AAA
 - **Checkpoint:** entiendes la correspondencia 1:1 de conceptos entre Mockito y Moq, y por qué en este proyecto se testea el Servicio en vez del Repositorio.
-

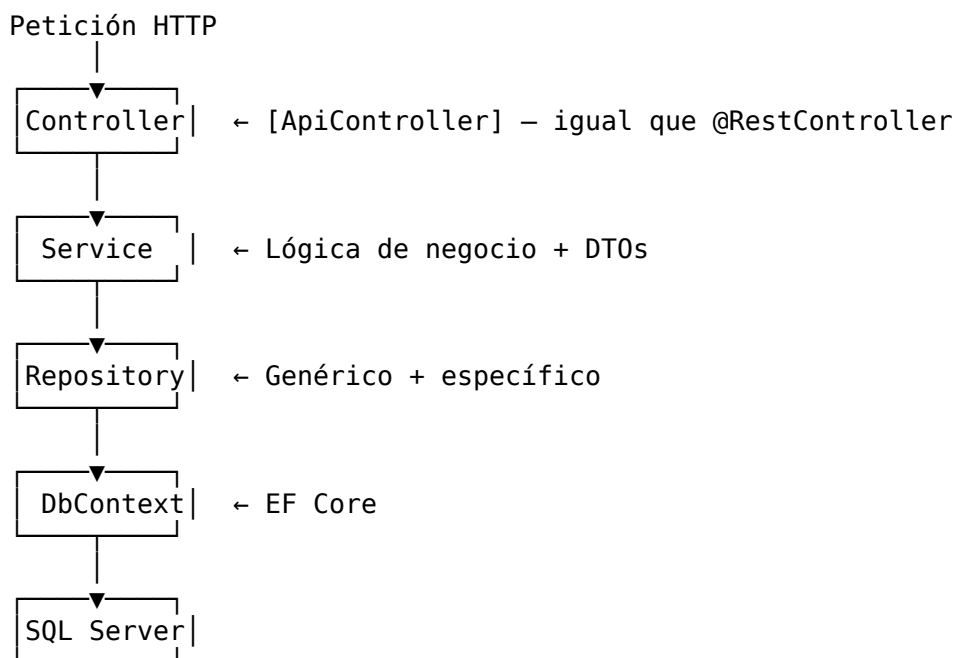
9. ASP.NET Core Web API (vs Spring Boot)

[↑ Volver al índice](#)

9.1. El “gran cambio” de Java, aplicado también en C

Tu Cap. 10-11 mostraban cómo Spring Boot reducía drásticamente el código de acceso a datos. En C#, ASP.NET Core + EF Core llega aún más lejos gracias a los repositorios genéricos: no hay clase equivalente a `BarcoDAOImpl` en absoluto — solo interfaces pequeñas con las consultas específicas.

9.2. Arquitectura en capas — la misma que ya conoces



Exactamente la misma arquitectura de tu Cap. 12.1 — Controller nunca toca el repositorio directamente, solo el Servicio.

9.3. Ejercicio

1. Dibuja el mismo diagrama de capas pero anotando, junto a cada caja, el archivo real del proyecto que la implementa
2. Explica en dos frases por qué el Controller **no debería** inyectar `IBarcoRepository` directamente (misma razón que en Java, Cap. 12.1)

- **Checkpoint:** la arquitectura en capas es idéntica conceptualmente; lo que cambia es cuánto código hace falta para llegar a ella.
-

10. Capa de Servicio

[↑ Volver al índice](#)

10.1. @Service → clase C# + registro explícito

```
@Service
public class BarcoService {
    [Autowired]
    private BarcoRepository barcoRepository;
}

// Ver archivo del proyecto: Services/BarcoService.cs
public class BarcoService : IBarcoService
{
    private readonly IBarcoRepository _barcoRepository;

    public BarcoService(IBarcoRepository barcoRepository, ILogger<BarcoService> logger)
    {
        _barcoRepository = barcoRepository;
    }
}
```

Buena práctica: en Java, tu Cap. 12.5 ya recomendaba inyección por constructor **en vez de** `@Autowired` sobre campo — en C# esa recomendación no es opcional, es la única forma que existe. Los campos son `readonly`, así que una vez inyectados no pueden cambiar accidentalmente.

10.2. @Transactional → SaveChangesAsync() (o transacción explícita)

```
@Transactional
public BarcoDTO save(BarcoDTO dto) {
    Barco barco = BarcoMapper.toEntity(dto);
    return BarcoMapper.toDTO(barcoRepository.save(barco));
}

// Ver archivo del proyecto: Services/BarcoService.cs
public async Task<BarcoDto> CreateAsync(BarcoRequestDto dto, CancellationToken ct = default)
{
    var barco = dto.ToEntity();
    var creado = await _barcoRepository.AddAsync(barco, ct);
    return creado.ToDto();
}
```

No hay un atributo equivalente a `@Transactional` que se ponga “por costumbre” en cada método: en EF Core, `SaveChangesAsync()` ya es transaccional por sí solo. Solo usamos una transacción explícita (`BeginTransactionAsync`) cuando de verdad agrupamos varias operaciones — ver `RegataService.DeleteAsync` en el proyecto, equivalente mejorado del `deleteWithCleanup` de tu Cap. 12.7.

10.3. Optional<T> → nullable reference types

```
public BarcoDTO findById(Long id) {
    return barcoRepository.findById(id)
        .map(BarcoMapper::toDTO)
        .orElse(null);
}

public async Task<BarcoDto> FindByIdAsync(long id, CancellationToken ct = default)
{
    var barco = await _barcoRepository.FindByIdAsync(id, ct)
        ?? throw new NotFoundException(nameof(Models.Barco), id);
    return barco.ToDto();
}
```

TIP: fíjate en la diferencia de filosofía: Java devolvía null y dejaba que el Controller comprobara `if (barco != null)` (Cap. 13.3). En C#, el Servicio **lanza una excepción de dominio** (`NotFoundException`) que un middleware global convierte en 404 — nadie más en la aplicación necesita comprobar null manualmente. Ver Capítulo 11 de esta guía.

10.4. AOP — @Transactional/@Autowired en Java vs Middleware en C

Tu Cap. 12.8 explicaba que `@Transactional` y `@Autowired` funcionan gracias a **AOP** (aspectos que se ejecutan automáticamente alrededor de tus métodos). En C#, el concepto equivalente son los **Middlewares** de ASP.NET Core (ver Capítulo 11) — cada petición HTTP pasa por una cadena de middlewares antes de llegar al Controller, cada uno pudiendo interceptar, modificar o cortocircuitar la petición.

10.5. Ejercicio

1. Compara `Services/RegataService.cs` con `RegataService.java` (Cap. 12.7): localiza el método `InscribirBarcoAsync` y explica con tus palabras por qué se modifica `barco.Regatas` y no `barco.Barcos` (pista: repasa el Capítulo 4 de esta guía sobre entidad propietaria)
2. Añade un método `FindTop5PorCapacidadAsync` al servicio de Barco
 - **Checkpoint:** entiendes que la capa de Servicio cumple la misma función en ambos lenguajes, y que el manejo de “no encontrado” cambia de un patrón null-check a excepciones + middleware.

11. API REST con Controllers

[↑ Volver al índice](#)

11.1. @RestController → [ApiController]

```
@RestController
@RequestMapping("/api/barcos")
public class BarcoController {
    @Autowired
    private BarcoService barcoService;
}
```

```
// Ver archivo del proyecto: Controllers/BarcosController.cs
[ApiController]
[Route("api/[controller]")]
public class BarcosController : ControllerBase
{
    private readonly IBarcoService _barcoService;

    public BarcosController(IBarcoService barcoService)
    {
        _barcoService = barcoService;
    }
}
```

[controller] en la ruta se sustituye automáticamente por el nombre de la clase sin el sufijo Controller — BarcosController → /api/Barcos.

11.2. Los 5 verbos, idénticos

Verbo	Java	C#
Listar	@GetMapping	[HttpGet]
Buscar uno	@GetMapping("/{id}") + @PathVariable	[HttpGet("{id:long}")] + parámetro
Crear	@PostMapping + @RequestBody	[HttpPost] + [FromBody]
Actualizar	@PutMapping("/{id}")	[HttpPut("{id:long}")]
Eliminar	@DeleteMapping("/{id}")	[HttpDelete("{id:long}")]

```
// Ver archivo del proyecto: Controllers/BarcosController.cs
[HttpGet("{id:long}")]
[ProducesResponseType(typeof(BarcoDto), StatusCodes.Status200OK)]
[ProducesResponseType(StatusCodes.Status404NotFound)]
public async Task<ActionResult<BarcoDto>> GetById(long id, CancellationToken ct) =>
    Ok(await _barcoService.FindByIdAsync(id, ct));
```

¿Sabías que? {id:long} en la ruta es una **restricción de tipo** — si alguien pide /api/barcos/abc, ASP.NET Core devuelve 404 automáticamente sin ni siquiera llegar a ejecutar el método, porque abc no es un long. Java no tiene un equivalente tan directo en @PathVariable.

11.3. ResponseEntity<T> → ActionResult<T>

```
return new ResponseEntity<>(barco, HttpStatus.OK);
return new ResponseEntity<>(HttpStatus.NOT_FOUND);
```

```
return Ok(barco);
return NotFound();
return CreatedAtAction(nameof(GetById), new { id = creado.Id }, creado); // 201 + Location h
return NoContent(); // 204
```

11.4. La mejora clave: sin if (x == null) en el Controller

Tu Cap. 13.3 repetía este patrón en **cada** endpoint:

```
@GetMapping("/{id}")
public ResponseEntity<BarcoDTO> getBarcoById(@PathVariable Long id) {
    BarcoDTO barco = barcoService.findById(id);
    if (barco != null) {
        return new ResponseEntity<>(barco, HttpStatus.OK);
    }
    return new ResponseEntity<>(HttpStatus.NOT_FOUND);
}
```

En el proyecto C#, ningún Controller tiene ese if:

```
// Ver archivo del proyecto: Controllers/BarcosController.cs
[HttpGet("{id:long}")]
public async Task<ActionResult<BarcoDto>> GetById(long id, CancellationToken ct) =>
    Ok(await _barcoService.FindByIdAsync(id, ct));
// Si no existe, el Servicio lanza NotFoundException.
// Un middleware global (ver Middleware/ExceptionHandlingMiddleware.cs)
// la convierte en 404 automáticamente. El Controller no necesita saberlo.
```

Este middleware es una pieza que **no tiene equivalente directo** en tu tutorial Java — es una mejora arquitectónica añadida específicamente en este port.

```
// Ver archivo del proyecto: Middleware/ExceptionHandlingMiddleware.cs
public class ExceptionHandlingMiddleware
{
    public async Task InvokeAsync(HttpContext context)
    {
        try { await _next(context); }
        catch (NotFoundException ex) { /* → 404 uniforme */ }
        catch (Exception ex) { /* → 500 uniforme, sin filtrar detalles internos */ }
    }
}
```

11.5. Ejercicio

1. Cuenta cuántas veces aparece `if (... == null)` en `Cap13_REST.md` de tu tutorial Java, y compáralo con `Controllers/BarcosController.cs` (debería ser cero)
2. Añade un endpoint `GET /api/barcos/eslora/{minima}` que use `FindByEsloraGreaterThanAsync`
 - **Checkpoint:** entiendes cómo el middleware centraliza el manejo de “no encontrado”, eliminando la repetición que existía en cada controlador Java.

12. DTOs y Mapper

[↑ Volver al índice](#)

12.1. El mismo problema, la misma solución

Tu Cap. 14 explicaba dos problemas de exponer entidades JPA directamente: bucle infinito por relaciones circulares y exposición de datos internos. **Exactamente el mismo problema existe en C#** con `System.Text.Json` — la solución es idéntica: DTOs.

12.2. record en vez de clase + Lombok

```
@Data
@NoArgsConstructor
@AllArgsConstructor
public class BarcoDTO {
    private Long id;
    private String nombre;
    private String tipo;
    private int eslora;
    private int manga;
    private int capacidad;
}
```

```
// Ver archivo del proyecto: Dtos/BarcoDtos.cs
public record BarcoDto(
    long Id,
    string Nombre,
    string Tipo,
    int Eslora,
    int Manga,
    int Capacidad
);
```

Una sola línea (record) sustituye a `@Data` + `@NoArgsConstructor` + `@AllArgsConstructor` de Lombok: genera constructor, igualdad por valor y `ToString()` automáticamente. Es sintaxis del propio lenguaje C#, no una librería añadida.

12.3. Mejora: DTO de entrada separado del de salida

Java reutilizaba el mismo `BarcoDTO` para request y response — lo que técnicamente permitía a un cliente **enviar un id** al crear un barco (aunque el servidor lo ignorase). En C# separamos:

```
// Ver archivo del proyecto: Dtos/BarcoDtos.cs
public record BarcoRequestDto(string Nombre, string Tipo, int Eslora, int Manga, int Capacidad)
// ← SIN Id: es físicamente imposible que el cliente lo envíe

public record BarcoDto(long Id, string Nombre, string Tipo, int Eslora, int Manga, int Capacidad)
// ← el de salida, CON Id (lo genera la BD)
```

12.4. BarcoMapper → extension methods

```
public class BarcoMapper {
    public static BarcoDTO toDTO(Barco barco) { ... }
    public static Barco toEntity(BarcoDTO dto) { ... }
}
```

```
// Uso: BarcoMapper.toDTO(barco)
```

```
// Ver archivo del proyecto: Mapping/BarcoMapper.cs
public static class BarcoMapper
{
    public static BarcoDto ToDto(this Barco barco) => new(...);
    public static Barco ToEntity(this BarcoRequestDto dto) => new() { ... };
}
```

```
// Uso: barco.ToDto() ← se lee como si fuera un método de la propia clase Barco
```

TIP: el `this` antes del primer parámetro convierte el método estático en un **extension method** — puedes escribir `barco.ToDto()` en vez de `BarcoMapper.ToDto(barco)`. Esto se encadena de maravilla con LINQ: `barcos.Select(b => b.ToDto()).ToList()`, exactamente igual que `barcos.stream().map(BarcoMapper::toDT0).collect(toList())` en Java (Cap. 14.6), pero sin necesidad de *method reference* explícito.

12.5. Ejercicio

1. Compara Mapping/BarcoMapper.cs con BarcoMapper.java (Cap. 14.4): identifica qué método sustituye a `updateEntityFromDT0`
 2. Crea `TripulanteDto` y `TripulanteMapper` para la entidad que añadiste en el ejercicio del Capítulo 3
- **Checkpoint:** entiendes por qué separar DTO de entrada y de salida es una mejora de seguridad sobre el `BarcoDT0` único de Java, y cómo `record` + `extension methods` sustituyen a Lombok + una clase Mapper tradicional.

13. Swagger / OpenAPI

[↑ Volver al índice](#)

13.1. Springdoc vs Swashbuckle

Java	C#
Librería	springdoc-openapi-starter-webmvc-ui
URL de la UI	/swagger-ui.html
Configuración mínima	application.properties (2 líneas)

13.2. Menos anotaciones necesarias

Java necesitaba anotar cada endpoint manualmente (Cap. 15.3-15.4): `@Tag`, `@Operation`, `@ApiResponse`, `@Parameter`. En C#, Swashbuckle **infiere automáticamente** mucho de esto a partir de los tipos:

```
// Ver archivo del proyecto: Controllers/BarcosController.cs
[HttpGet("{id:long}")]
[ProducesResponseType(typeof(BarcoDto), StatusCodes.Status200OK)]
[ProducesResponseType(StatusCodes.Status404NotFound)]
public async Task<ActionResult<BarcoDto>> GetById(long id, CancellationToken ct) => ...
```

Con solo `[ProducesResponseType]` (que además sirve para documentar, no solo para Swagger), ya se genera automáticamente en la UI: el tipo de respuesta, los códigos posibles y los parámetros — sin `@Operation` ni `@ApiResponse` explícitos como en Java.

13.3. Configuración en Program.cs

```
// Ver archivo del proyecto: Program.cs
builder.Services.AddEndpointsApiExplorer();
builder.Services.AddSwaggerGen(options =>
{
    options.SwaggerDoc("v1", new() { Title = "Marina API", Version = "v1" });
});

// más abajo:
app.UseSwagger();
app.UseSwaggerUI();
```

13.4. Ejercicio

1. Arranca la API y abre /swagger — compara visualmente con las capturas descritas en tu Cap. 15.5
2. Prueba el endpoint POST /api/regatas/{regataId}/barcos/{barcoId} (inscribir barco) desde la propia UI de Swagger, con “Try it out”
 - **Checkpoint:** Swashbuckle cubre el mismo rol que Springdoc, generando más documentación automáticamente a partir del código, con menos anotaciones manuales.

14. Probar la API: Swagger UI y Postman

[↑ Volver al índice](#)

14.1. Todo lo de tu Cap. 16 se aplica igual

Postman **no distingue** si el backend es Java o C# — habla HTTP, y tanto Spring Boot como ASP.NET Core exponen el mismo tipo de endpoints REST. Puedes reutilizar exactamente la misma colección de Postman de tu Cap. 16, cambiando solo el puerto (ASP.NET Core suele usar un puerto distinto a 8080, indicado en la consola al arrancar con `dotnet run`).

14.2. Diferencias menores a tener en cuenta

Aspecto	Java (Cap. 16)	C#
Puerto por defecto	8080	Variable (aparece en consola al hacer <code>dotnet run</code> , típicamente 5000/5001 o el que asigne <code>launchSettings.json</code>)
Formato de error	Cuerpo vacío en 404	JSON <code>problem+json</code> con <code>title</code> , <code>status</code> , <code>detail</code> (RFC 7807) — más información útil para depurar desde Postman
Scripts de test (<code>pm.test(...)</code>)	Igual	Igual — el JavaScript de Postman no cambia

14.3. Ejercicio

1. Importa/recrea la colección de tu Cap. 16.3 apuntando al puerto de la API C#

2. Prueba un GET /api/barcos/999 (id inexistente) y observa el cuerpo problem+json de la respuesta — compáralo con el cuerpo vacío que devolvía Java en el mismo caso
- **Checkpoint:** sabes que las herramientas de prueba (Postman, Swagger UI, curl) son agnósticas al lenguaje del backend — lo único que cambia es el formato de los errores, más informativo en este proyecto C#.

15. Resumen: mapa completo del proyecto

[↑ Volver al índice](#)

15.1. Tabla maestra de equivalencias

Concepto	Java (tu tutorial)	C# (proyecto MarinaApi)
ORM	Hibernate	Entity Framework Core
Especificación	JPA	(no aplica, EF Core es único)
Contexto de persistencia	Session + SessionFactory	DbContext
Configuración BD	hibernate.cfg.xml / application.properties	appsettings.json
Mapeo entidad-tabla	@Entity, @Column	Convención + Data Annotations
Relaciones	@OneToOne, @ManyToOne, mappedBy	Fluent API en OnModelCreating
Getters/setters	Lombok @Data	Propiedades automáticas nativas
DAO	BarcoDAO + BarcoDAOImpl (uno por entidad)	GenericRepository<T> único
Consultas	HQL / SQL nativo / Critería API	LINQ (unifica los tres)
Framework web	Spring Boot	ASP.NET Core
Inyección de dependencias	@Autowired, auto-detectado	Constructor + registro explícito en Program.cs
Capa de negocio	@Service	Clase de servicio + interfaz
Transacciones	@Transactional	SaveChangesAsync() implícito / BeginTransactionAsync() explícito
Controlador REST	@RestController	[ApiController]
DTOs	Clase + Lombok	record
Mapper	Clase con métodos estáticos	Extension methods
Manejo de “no encontrado”	if (x == null) return 404 en cada método	Excepción + Middleware global
Testing	JUnit 5 + Mockito	xUnit + Moq + FluentAssertions
Documentación API	Springdoc OpenAPI	Swashbuckle
Motor de BD	MySQL (tutorial)	SQL Server (real, Espiral MS)

15.2. Cómo sigue encajando con tu plan de verano

- Este proyecto es la aplicación práctica de las **Lecciones 6-8** de tu GUIA_DEFINITIVA_CSHARP.md (Patrones de Diseño, SQL, Entity Framework Core)
- Úsalo para practicar la **Lección 9** (Git): crea ramas feature/tripulantes siguiendo los ejercicios de esta guía, haz commits con Conventional Commits, simula un PR

- Úsalo para practicar la **Lección 10** (Scrum): trocea los ejercicios de cada capítulo de este documento en historias de usuario con story points, y corre un mini-sprint tú mismo

15.3. Qué NO se ha portado (y por qué no hace falta)

- **AOP personalizado** (Cap. 12.8 de Java): no lo necesitas — los middlewares de ASP.NET Core cubren el mismo rol para lo que este proyecto necesita
- **Thymeleaf** (mencionado en el roadmap de tu Cap. 1): si las prácticas requieren interfaz web server-side en vez de solo API REST, el equivalente sería Razor Pages — pídemelo aparte si llega a hacer falta

Fin de la guía.

Con esto tienes tanto la teoría (este documento) como la práctica ya resuelta (el proyecto MarinaApi.zip). Recomendación de uso: lee un capítulo, abre el archivo del proyecto que referencia, y solo después haz el ejercicio propuesto — igual que trabajabas con los capítulos Java originales.